

JAVA

02

RDBMS & Database Programming With JDBC

Connect your Java code to the real world — master databases, queries, and persistent data with JDBC.

DURATION

6 Sessions

LEVEL

Intermediate

MODULE

Module 2

UPDATED

June 2026

IN THIS MODULE

01

Introduction to JDBC

02

JDBC Setup & Connection

03

Statement & PreparedStatement

04

Select Queries & ResultSet

05

Transactions & Batch Processing

06

Integration & Error Handling

SESSION

01

60 MIN · FUNDAMENTALS

Introduction to JDBC

◆ Learning Objectives

- Explain what JDBC is and why it is needed in Java applications
- Describe the JDBC architecture and its four core components
- Identify the four types of JDBC drivers and choose the right one
- Trace the data flow from a Java program to a MySQL database
- Import the correct JDBC packages in a Java class
- Understand how JDBC fits into the larger full-stack picture

01 Why JDBC? The Gap Between Java and Databases

Your Java program runs in memory — it can add, subtract, and process data. But the moment your program closes, all that data disappears. A database like MySQL keeps data alive even after the JVM shuts down. JDBC (Java Database Connectivity) is the bridge that lets your Java code talk directly to any relational database, send SQL queries, and get results back — all from within the same program.

CONCEPT — What is JDBC?

JDBC is a standard Java API (part of the `java.sql` package) that defines how Java programs connect to relational databases. Think of it as a universal remote control — whether your database is MySQL, Oracle, or PostgreSQL, the Java code you write stays almost identical. Only the driver (the 'battery') changes. In India, every major banking and fintech backend — from SBI's core banking to PhonePe's wallet — uses JDBC or a framework built on top of it.

02 JDBC Architecture — Four Core Components

Component	What It Does	Real-World Analogy
DriverManager	Loads the driver and creates a connection	Receptionist who connects your call
Connection	Represents an open channel to the database	The phone line that stays open
Statement / PreparedStatement	Sends SQL queries through the connection	The words you speak into the phone
ResultSet	Holds the rows returned by a SELECT query	The answer you hear back

These four components work in sequence — you always follow the same flow: load driver -> get connection -> create statement -> execute query -> read ResultSet -> close everything. Skipping any step causes either a runtime error or a resource leak that quietly slows your server over time.

03 JDBC Driver Types

Type 1 & Type 2 (Legacy)

Require native OS libraries or ODBC bridges. Rarely used today. Slower, platform-dependent, and hard to deploy on cloud servers.

Type 4 — Pure Java Driver (Industry Standard)

Converts JDBC calls directly to the database's native protocol. No extra software needed. MySQL Connector/J is a Type 4 driver — download one JAR and you are ready.

PRO TIP — Always Use Type 4

In every real project — startups, banks, e-commerce — you will use a Type 4 Pure Java driver. It is portable, works on any OS, and ships as a single JAR file. For MySQL, the driver class is `com.mysql.cj.jdbc.Driver` (Connector/J 8.x). Memorise this string — you will type it hundreds of times in your career.

04 The Java -> MySQL Data Flow

Picture a user on a PhonePe-style app tapping 'Send Money'. At the backend, a Java program creates a JDBC Connection to the MySQL database, fires an UPDATE query to deduct balance from the sender's account, fires another UPDATE to credit the receiver, and then commits both changes as one atomic transaction. The ResultSet confirms success, and the API sends back a 200 OK response. Every step travels through JDBC.

<code>import java.sql.Connection;</code>	Represents the DB connection
<code>import java.sql.DriverManager;</code>	Factory class that creates connections
<code>import java.sql.Statement;</code>	Executes plain SQL strings
<code>import java.sql.PreparedStatement;</code>	Executes parameterized SQL safely
<code>import java.sql.ResultSet;</code>	Holds rows returned by SELECT
<code>import java.sql.SQLException;</code>	Must always be caught or declared

TRY THIS LIVE

Open your IDE and create a new class called `JDBCDemo`. Add all six import statements above. Now hover over `Connection` in IntelliJ — you will see it is an interface, not a class. This means MySQL, Oracle, and PostgreSQL all implement the same interface, which is exactly why your code stays portable. Notice how `DriverManager` is the only concrete class you touch directly.

05 Where JDBC Fits in the Full Stack

JAVA BACKEND MARKET SHARE

~30%

Source: Stack Overflow Developer Survey 2024 — Java remains among the top server-side languages globally

CASE STUDY — IRCTC Ticket Booking

Context — IRCTC (Indian Railway Catering and Tourism Corporation) processes over 8 lakh ticket bookings per day on its Java-based backend.

Challenge — At peak times — Tatkal opening at 10 AM — thousands of users simultaneously try to book the same seat. The system must update seat availability and charge payment atomically, with zero double-booking.

What they did — IRCTC's backend uses JDBC-based connection pooling (via frameworks like Hibernate on top of JDBC) to open thousands of concurrent database connections efficiently, executing UPDATE and INSERT statements wrapped in transactions.

Result — The system handles millions of seat-lock and booking operations per hour without data corruption — a direct result of JDBC's connection and transaction model.

Takeaway — JDBC's architecture — connection, statement, transaction — is not just academic. It is the foundation holding India's busiest ticketing platform together.

Source: IRCTC Annual Report 2023-24 / publicly reported system architecture

“JDBC is not just an API — it is the handshake between your logic and your data. Master this handshake and every framework on top of it (Hibernate, Spring Data JPA) becomes easy to understand.”

WHERE TO GO NEXT

You now know what JDBC is and how its four components fit together. Next session you will actually download the driver JAR and write your first live connection.

Explore deeper — Official JDBC documentation:
<https://docs.oracle.com/javase/tutorial/jdbc/overview/index.html>

Try on your own — Draw the Java -> DriverManager -> Connection -> Statement -> ResultSet flow on paper. Add a MySQL cylinder at the end. This diagram will be your mental model for the next five sessions.

Connect the dots — Session 02 picks up exactly where this ends — you will install MySQL Connector/J and run `DriverManager.getConnection()` for the first time.

Going further — Read about JDBC 4.0's automatic driver loading (`Class.forName()` is no longer needed in modern Java) — search 'JDBC 4.0 automatic driver discovery'.

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Week 9, Lecture 1: Introduction to JDBC

Link — <https://nptel.ac.in/courses/106105191>

Always link the official channel — beware fan-made or re-upload copies.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java trainer. Explain JDBC architecture in simple Hinglish using an analogy of ordering food at a restaurant — the waiter is DriverManager, the kitchen is MySQL, and the menu is the SQL query. Keep it under 150 words and give one Indian real-world example.

SESSION

02

60 MIN · SETUP & CONFIGURATION

JDBC Setup & Connecting to MySQL

◆ Learning Objectives

- Download and add the MySQL Connector/J JAR to a Java project
- Write a working JDBC connection using `DriverManager.getConnection()`
- Identify the three parameters of a JDBC connection string (URL, user, password)
- Handle `SQLException` correctly in a connection block
- Verify a successful connection by printing metadata to the console
- Avoid the most common setup mistakes that cause `ClassNotFoundException` and connection refused errors

01 What You Need Before Writing a Single Line

Prerequisite	Version to Use	Where to Get It
JDK	17 or 21 (LTS)	https://adoptium.net
MySQL Server	8.0+	https://dev.mysql.com/downloads/mysql/
MySQL Connector/J (JAR)	8.x (mysql-connector-j-8.x.x.jar)	https://dev.mysql.com/downloads/connector/j/
IDE	IntelliJ IDEA Community or Eclipse	https://www.jetbrains.com/idea/

The MySQL Connector/J JAR is the Type 4 JDBC driver for MySQL. It is a single file — roughly 2 MB — that contains everything Java needs to speak MySQL's native protocol. You do not install it system-wide; you add it to your project's classpath, and your code finds it automatically at runtime.

02 Adding the JAR to Your Project

CONCEPT — Classpath

The classpath is the list of folders and JAR files Java searches when loading classes. If `mysql-connector-j-8.x.x.jar` is not on the classpath, the JVM cannot find the driver class and throws `ClassNotFoundException` at runtime. In IntelliJ: File -> Project Structure -> Modules -> Dependencies -> + -> JARs or Directories -> select your JAR. In Maven projects, add the dependency in `pom.xml` and the IDE handles the rest.

<code><!-- Maven pom.xml dependency --></code>	Add inside <dependencies>
<code><dependency></code>	Opening tag
<code><groupId>com.mysql</groupId></code>	MySQL group
<code><artifactId>mysql-connector-j</artifactId></code>	Connector artifact
<code><version>8.3.0</version></code>	Use latest stable 8.x version
<code></dependency></code>	Closing tag — Maven auto-downloads the JAR

03 The JDBC URL — Your Database Address

A JDBC URL works like a web URL — it tells DriverManager exactly which database server to reach, on which port, and which database to open. The format for MySQL is: `jdbc:mysql://hostname:port/databaseName?parameters`. On your local machine, hostname is localhost, port is 3306 (MySQL default), and dbName is whatever schema you created in MySQL Workbench.

CONCEPT — JDBC URL Breakdown

`jdbc:mysql://localhost:3306/expense_tracker?useSSL=false&serverTimezone=UTC`

- 'jdbc' tells Java to use JDBC.
- 'mysql' selects the MySQL driver.
- 'localhost:3306' is the server address and port.
- 'expense_tracker' is the database schema name.
- Parameters after '?' handle SSL and timezone — always include `serverTimezone=UTC` to avoid errors in Java 8+.

04 Writing Your First Connection

```
String url =
"jdbc:mysql://localhost:3306/expense_tracker?useSSL=false&serverTimezone=UTC";
    JDBC URL for local MySQL
String user = "root";
    MySQL username
String password = "your_password";
    MySQL password — never hardcode in production
Connection conn = DriverManager.getConnection(url, user, password);
    Opens the connection — throws SQLException if it fails
System.out.println("Connected: " + conn.getMetaData().getDatabaseProductVersion());
    Prints MySQL version as proof of live connection
conn.close();
    Always close the connection when done
```

TRY THIS LIVE

Open MySQL Workbench and create a schema called `expense_tracker` (run: `CREATE DATABASE`

expense_tracker;). Then run the code block above in your IDE. If you see 'Connected: 8.x.x' in the console, your full JDBC pipeline is working. If you see 'Connection refused', MySQL is not running — check Services on Windows or run 'sudo service mysql start' on Linux.

05 Handling SQLException Correctly

`DriverManager.getConnection()` declares that it throws `SQLException`. Unlike `NullPointerException` which is unchecked, `SQLException` is a checked exception — the compiler forces you to handle it. You must either surround the call with try-catch or declare throws `SQLException` on your method. In production code, always use try-catch so you can log the error and show a meaningful message instead of crashing.

```
try {
    Connection conn = DriverManager.getConnection(url, user, password);
    Attempt connection
    System.out.println("Connection successful!");
    Runs only if no exception
    conn.close();
    Release the connection
} catch (SQLException e) {
    Catch any DB error
    System.out.println("Error: " + e.getMessage());
    Print the MySQL error message
    System.out.println("SQLState: " + e.getSQLState());
    5-char code like '08001' for connection failure
}
End of try-catch
```

WARNING — Never Hardcode Passwords

Hardcoding database credentials in Java source files is a critical security risk — if the code is ever pushed to GitHub, your database is exposed to the internet within minutes. Always read credentials from environment variables or a properties file that is excluded from version control via `.gitignore`. This is the real-world practice from day one of your first job.

06 Common Errors and How to Fix Them

Error Message	Most Likely Cause	Fix
<code>ClassNotFoundException:</code> <code>com.mysql.cj.jdbc.Driver</code>	JAR not on classpath	Add <code>mysql-connector-j</code> JAR to project dependencies
Communications link failure	MySQL server not running or wrong port	Start MySQL service; verify port 3306

Error Message	Most Likely Cause	Fix
Access denied for user 'root'@'localhost'	Wrong username or password	Reset MySQL root password or use correct credentials
Unknown database 'expense_tracker'	Schema not created yet	Run CREATE DATABASE expense_tracker; in MySQL Workbench

CASE STUDY — Zepto's Backend Setup

Context — Zepto, the Mumbai-based 10-minute grocery delivery startup (valued at \$5 billion in 2024), runs a Java/Spring Boot backend that connects to multiple MySQL-compatible databases.

Challenge — During rapid scaling from 10 to 100+ dark stores in 2022, Zepto's engineers discovered that naive `DriverManager.getConnection()` calls — one per request — were exhausting database connections under high load, causing order failures.

What they did — They replaced direct `DriverManager` calls with a connection pool (HikariCP), which pre-creates a fixed number of JDBC Connection objects and reuses them across requests. The JDBC URL and credentials were moved to environment variables in their Docker containers.

Result — Connection failures dropped to near zero even at 50,000 simultaneous orders. The key insight was that a Connection is expensive to create — pooling them is essential at scale.

Takeaway — `DriverManager.getConnection()` is the learning tool; connection pooling is how production systems work. Understand the basics first, then you will appreciate why pools exist.

Source: Zepto engineering blog / YourStory coverage, 2023

“A JDBC connection is like a taxi ride — it costs money to start. In production, you keep a fleet of taxis (connection pool) always running so customers do not wait.”

WHERE TO GO NEXT

You can now open a live JDBC connection to MySQL. Next, you will learn how to actually send SQL queries through that connection using `Statement` and `PreparedStatement`.

Explore deeper — MySQL Connector/J configuration guide:

<https://dev.mysql.com/doc/connector-j/8.0/en/connector-j-reference-configuration-properties.html>

Try on your own — Create a second schema called `tops_demo` and modify your connection URL to point to it. Confirm the connection prints the correct database name using `conn.getCatalog()`.

Connect the dots — Session 03 uses the same Connection object you just created — you will pass it to a `PreparedStatement` and run your first INSERT query.

Going further — Research HikariCP — the fastest JDBC connection pool in Java. It is the default pool in Spring Boot. Understanding why it beats C3P0 and DBCP will make you stand out in interviews.

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)**Channel** — NPTEL (official — IITs / IISc)**Watch** — Programming in Java — Week 9, Lecture 2: JDBC — Connecting to Database**Link** — <https://nptel.ac.in/courses/106105191>**AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE**

Act as a senior Java developer. Explain in simple Hinglish why hardcoding a database password in Java source code is dangerous. Use the example of a Surat-based startup whose GitHub repo was found public by mistake. Suggest the correct alternative using environment variables. Keep it under 150 words.

SESSION

03

60 MIN · DATA MANIPULATION

Statement & PreparedStatement

◆ Learning Objectives

- Distinguish between Statement and PreparedStatement and know when to use each
- Write INSERT, UPDATE, and DELETE queries using PreparedStatement
- Use setString(), setInt(), setDouble() to bind parameters safely
- Explain SQL Injection with a real example and demonstrate how PreparedStatement prevents it
- Interpret the int return value of executeUpdate()
- Structure JDBC data-manipulation code using proper try-catch-finally patterns

01 Statement vs PreparedStatement — The Core Difference

Statement

Builds the SQL string by concatenating Java variables directly. Simple to write but vulnerable to SQL Injection. Recompiled by MySQL every time it runs. Fine for static queries with no user input (e.g., schema creation scripts).

PreparedStatement

Sends a query template with '?' placeholders to MySQL first. MySQL compiles it once. You then bind real values using setter methods. Safe from SQL Injection, faster for repeated queries, and cleaner to read. Use this for all queries that include user data.

In practice, you will use PreparedStatement for nearly every query your application runs. Statement is useful only for DDL commands (CREATE TABLE, DROP TABLE) that you run once from a setup script and never from user input. The moment user-supplied text enters a query — a name, an amount, an email — PreparedStatement is non-negotiable.

02 SQL Injection — The Most Dangerous JDBC Mistake

WARNING — SQL Injection Attack

Imagine a login form where a user enters their email. With Statement, a hacker types: ' OR '1'=1' as the email. The concatenated query becomes: SELECT * FROM users WHERE email=" OR '1'=1' — which returns ALL users and bypasses login completely. This attack has been used to steal millions of records from Indian government portals and e-commerce sites. PreparedStatement makes this impossible because user input is never treated as SQL code.

```
// UNSAFE — Never do this with user input
String email = request.getParameter("email");
```

SQL Injection vulnerable
Could be malicious input

```
String sql = "SELECT * FROM users WHERE email='" + email + "'";
```

Concatenation — dangerous

```
Statement st = conn.createStatement();
```

Plain Statement

```
ResultSet rs = st.executeQuery(sql);
```

Executes the injected SQL

03 Writing INSERT with PreparedStatement

The '?' character is a positional placeholder. The first '?' is index 1, the second is index 2, and so on. After creating the PreparedStatement, you call setter methods — setString(), setInt(), setDouble() — to bind the actual values. MySQL receives the structure and the values separately, which is why injection becomes impossible: the values are data, never code.

```
String sql = "INSERT INTO expenses (user_id, category, amount, note) VALUES (?, ?, ?, ?)";
```

4 placeholders — 4 setters needed

```
PreparedStatement ps = conn.prepareStatement(sql);
```

MySQL compiles the template

```
ps.setInt(1, 101);
```

Bind user_id = 101 at position 1

```
ps.setString(2, "Food");
```

Bind category = 'Food' at position 2

```
ps.setDouble(3, 250.50);
```

Bind amount = 250.50 at position 3

```
ps.setString(4, "Dinner at Gujarat Thali");
```

Bind note at position 4

```
int rows = ps.executeUpdate();
```

Returns number of rows affected (should be 1)

```
System.out.println(rows + " row(s) inserted.");
```

Confirm the insert succeeded

TRY THIS LIVE

In your expense_tracker database, run this SQL in MySQL Workbench to create the table: CREATE TABLE expenses (id INT AUTO_INCREMENT PRIMARY KEY, user_id INT, category VARCHAR(50), amount DECIMAL(10,2), note VARCHAR(255)); Then run the INSERT code above in Java. After it prints '1 row(s) inserted', switch back to MySQL Workbench and run SELECT * FROM expenses; to see your data.

04 UPDATE and DELETE with PreparedStatement

```
// UPDATE — change the amount of a specific expense
```

Pattern: SET first, WHERE last

```
String updateSql = "UPDATE expenses SET amount = ? WHERE id = ?";
```

2 placeholders

```
PreparedStatement psUpdate = conn.prepareStatement(updateSql);
```

Prepare the update

```
psUpdate.setDouble(1, 300.00);
```

New amount value

```
psUpdate.setInt(2, 1);
```

ID of the row to update

```
int updated = psUpdate.executeUpdate();
```

Returns 1 if found and updated, 0 if ID not found

```
// DELETE — remove an expense by ID
```

Always use WHERE — without it you delete ALL rows

```
String deleteSql = "DELETE FROM expenses WHERE id = ?";
```

One placeholder

```
PreparedStatement psDelete = conn.prepareStatement(deleteSql);
```

Prepare the delete

```
psDelete.setInt(1, 1);
```

ID of the expense to remove

```
int deleted = psDelete.executeUpdate();
```

Returns 1 if deleted, 0 if not found

CONCEPT — executeUpdate() Return Value

executeUpdate() returns an int — the number of rows affected. For INSERT, it is usually 1. For UPDATE and DELETE, it tells you how many rows matched your WHERE clause. If it returns 0, no rows matched — which usually means the WHERE condition was wrong. Always check this return value in production code; silently ignoring a 0 is how bugs stay hidden for months.

05 Reusing a PreparedStatement for Multiple Rows

One of PreparedStatement's biggest advantages is that once prepared, you can rebind parameters and execute again without re-sending the SQL template to MySQL. This is significantly faster when inserting or updating many rows in a loop — the database engine reuses its compiled execution plan each time.

```
String sql = "INSERT INTO expenses (user_id, category, amount) VALUES (?, ?, ?)";
```

Prepared once

```
PreparedStatement ps = conn.prepareStatement(sql);
```

Compiled once by MySQL

```
String[][] data = {"Food", "120"}, {"Travel", "450"}, {"Grocery", "890"};
```

Sample data

```
for (String[] row : data) {
```

Loop over each row

```
    ps.setInt(1, 101);
```

Same user_id each time

```
    ps.setString(2, row[0]);
```

Category from array

```
ps.setDouble(3, Double.parseDouble(row[1]));
```

Amount from array

```
    ps.executeUpdate();
```

Executes with the new values

```
}
```

SQL template was compiled only once

Method

Used For

Returns

Method	Used For	Returns
<code>executeUpdate()</code>	INSERT, UPDATE, DELETE	int — rows affected
<code>executeQuery()</code>	SELECT	ResultSet — contains matching rows
<code>execute()</code>	Any SQL — use when unsure	boolean — true if ResultSet, false if row count

CASE STUDY — Paytm Merchant Payouts

Context — Paytm processes merchant settlement payouts to lakhs of small shop owners across Gujarat, Rajasthan, and UP — many running kirana stores and tea stalls.

Challenge — Each payout is a database record: merchant ID, amount, timestamp, status. Generating settlement records for 5 lakh merchants nightly using Statement with string concatenation was not only slow — the SQL strings were so long and complex that even a minor parsing error could corrupt a payout batch.

What they did — Paytm's settlement engine was rewritten to use PreparedStatement in a loop with connection pooling. Each payout record was bound via `setLong()`, `setBigDecimal()`, and `setTimestamp()` — no string concatenation anywhere in the query path.

Result — Payout batch processing time dropped significantly. More importantly, injection risk was eliminated, and the code became auditable — critical for RBI compliance as a payment aggregator.

Takeaway — PreparedStatement is not a nice-to-have — in financial systems it is a regulatory requirement. RBI guidelines mandate that all transaction-touching code be SQL-injection proof.

Source: Paytm engineering blogs / RBI Payment Aggregator Guidelines 2022

“If your query has a '?' in it, you are safe. If it has a '+' to add user input, you are vulnerable. It is that simple.”

WHERE TO GO NEXT

You can now write, execute, and verify INSERT, UPDATE, and DELETE safely with PreparedStatement. The next step is pulling data back out with SELECT queries and ResultSet.

Explore deeper — OWASP SQL Injection Prevention Cheat Sheet:

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

Try on your own — Write a method `updateExpenseCategory(int id, String newCategory)` that takes an expense ID and a new category string and updates the row. Test it with a valid ID and then with an ID that does not exist — observe the different return values.

Connect the dots — Session 04 introduces `executeQuery()` and `ResultSet` — reading data back from the database to display to users.

Going further — Research 'JDBC Named Parameters' — a pattern where you use `:name` instead of `?` for readability. Spring JDBC's `NamedParameterJdbcTemplate` implements this. Understanding why it was built will deepen your appreciation of `PreparedStatement`'s design.

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Week 9, Lecture 3: JDBC — Executing SQL Statements

Link — <https://nptel.ac.in/courses/106105191>

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a cybersecurity trainer. Explain SQL Injection in simple Hinglish using the example of a student entering their name on a Gujarat university portal and breaking the database. Then show how `PreparedStatement` fixes it. Keep it under 150 words and use one specific code example.

SESSION

04

60 MIN · DATA RETRIEVAL

Select Queries & ResultSet

◆ Learning Objectives

- Execute a SELECT query using PreparedStatement and executeQuery()
- Traverse a ResultSet using the while(rs.next()) pattern
- Read column values using getString(), getInt(), getDouble(), and getTimestamp()
- Display query results in a formatted console table
- Implement pagination using SQL LIMIT and OFFSET
- Sort results dynamically using ORDER BY in a PreparedStatement

01 From INSERT to SELECT — Reading Data Back

In the previous session you pushed data into the database. Now you need to pull it back out and show it to the user. This is the most frequent operation any application performs — a typical expense tracker app does 10 SELECT queries for every 1 INSERT. Understanding how to read a ResultSet efficiently is one of the most important skills in backend development.

CONCEPT — ResultSet Cursor

A ResultSet works like a pointer (cursor) that starts just before the first row. Each call to `rs.next()` moves the pointer one row forward and returns true if a row exists, or false when there are no more rows. This is why the standard pattern is `while(rs.next())` — it keeps reading rows until it runs out. Think of it like reading a ledger book page by page: you cannot read all pages at once; you turn one page at a time.

02 Executing a SELECT Query

```
String sql = "SELECT id, category, amount, note FROM expenses WHERE user_id = ?";
```

Filter by user_id for safety

```
PreparedStatement ps = conn.prepareStatement(sql);
```

Prepare the SELECT

```
ps.setInt(1, 101);
```

Bind user_id = 101

```
ResultSet rs = ps.executeQuery();
```

Returns ResultSet — never null, but may be empty

```
while (rs.next()) {
```

Loop — moves cursor to next row, returns false when done

```
    int id = rs.getInt("id");
```

Read int column by name

```
    String cat = rs.getString("category");
```

Read String column by name

```
    double amt = rs.getDouble("amount");
```

Read decimal column

```
    String note = rs.getString("note");
```

Read note column

```
System.out.printf("%-5d %-12s Rs %-10.2f %s\n", id, cat, amt, note);
    Formatted output with Rs symbol
}
```

End of while loop

TRY THIS LIVE

Run the code above after inserting at least 3 rows in Session 03. You should see a formatted table in your console. Now modify the query to add ORDER BY amount DESC — observe how the most expensive item moves to the top. Then try changing the user_id to 999 (no such user) and confirm that the while loop simply does not execute — this is the safe 'zero results' behaviour.

03 ResultSet Getter Methods — Complete Reference

Method	Java Type Returned	Typical SQL Column Type
rs.getInt("col")	int	INT, TINYINT, SMALLINT
rs.getLong("col")	long	BIGINT, AUTO_INCREMENT large IDs
rs.getDouble("col")	double	DECIMAL, FLOAT, DOUBLE
rs.getString("col")	String	VARCHAR, CHAR, TEXT
rs.getBoolean("col")	boolean	TINYINT(1) used as boolean in MySQL
rs.getTimestamp("col")	java.sql.Timestamp	DATETIME, TIMESTAMP
rs.getBigDecimal("col")	BigDecimal	DECIMAL — use for financial amounts, never double

PRO TIP — Use BigDecimal for Money

getDouble() has floating-point precision issues. Rs 250.10 stored as double can come back as 250.09999999998 due to binary rounding — a nightmare for financial reports. Always use getBigDecimal('amount') when reading monetary values. Every bank, every payment app, every accounting system in India uses BigDecimal for rupee amounts.

04 Displaying Formatted Output

Console output looks professional when columns are aligned. Java's System.out.printf() with format specifiers makes this easy. %-15s means left-aligned string in 15 characters wide. %10.2f means right-aligned decimal with 2 decimal

places in 10 characters. Adding a header row with a separator line transforms raw output into a readable report — exactly what you would need for the Smart Expense Tracker mini project.

```
System.out.printf("%-5s %-15s %-12s %-30s\n", "ID", "CATEGORY", "AMOUNT", "NOTE");
    Header row
System.out.println("-".repeat(65));           Separator line — 65 dashes
while (rs.next()) {                          Loop through rows
    System.out.printf("%-5d %-15s Rs %-11.2f %-30s\n",
        Format: ID, category, rupee amount, note
        rs.getInt("id"), rs.getString("category"),
        ID and category
        rs.getDouble("amount"), rs.getString("note"));
        Amount and note
    }
    End loop
```

05 Pagination — Showing Results Page by Page

No application shows all database rows at once. A user with 500 expenses sees 10 per page. SQL's LIMIT and OFFSET clauses implement this. LIMIT is how many rows to return. OFFSET is how many rows to skip. Page 1 is LIMIT 10 OFFSET 0. Page 2 is LIMIT 10 OFFSET 10. Page 3 is LIMIT 10 OFFSET 20. The formula is: offset = (pageNumber - 1) × pageSize.

```
int pageSize = 10;           Rows per page
int pageNumber = 2;         Which page the user wants
int offset = (pageNumber - 1) * pageSize;   = 10 for page 2
String sql = "SELECT * FROM expenses WHERE user_id = ? ORDER BY id DESC LIMIT ? OFFSET ?";
    Paginated query, newest first
PreparedStatement ps = conn.prepareStatement(sql);
    Prepare
ps.setInt(1, 101);           user_id
ps.setInt(2, pageSize);       LIMIT = 10
ps.setInt(3, offset);         OFFSET = 10 for page 2
ResultSet rs = ps.executeQuery();
    Returns only 10 rows for page 2
```

OFFSET Pagination (Standard)

LIMIT 10 OFFSET 200. Simple to implement — add two parameters. Works well up to tens of thousands of rows. Becomes slow on very large tables because MySQL must count through 200 rows before returning 10. Fine for most applications and all mini projects.

Keyset Pagination (Advanced)

WHERE id > last_seen_id LIMIT 10. Jumps directly to the correct row using an index. No performance degradation on large tables. Used by Instagram, Twitter, and large-scale Indian apps like Swiggy order history. Covered in advanced SQL courses.

06 Example — Monthly Expense Report by Category

```
String sql = "SELECT category, SUM(amount) AS total FROM expenses"
```

Aggregate query

```
+ " WHERE user_id = ? AND MONTH(created_at) = ?"
```

Filter by user and month

```
+ " GROUP BY category ORDER BY total DESC";
```

Group by category, highest spend first

```
PreparedStatement ps = conn.prepareStatement(sql);
```

Prepare

```
ps.setInt(1, 101);
```

user_id

```
ps.setInt(2, 6);
```

Month = June

```
ResultSet rs = ps.executeQuery();
```

Run

```
while (rs.next()) {
```

Print each category total

```
System.out.printf("%-15s Rs %.2f\n", rs.getString("category"), rs.getDouble("total"));
```

Category and summed amount

```
}
```

End

CASE STUDY — Jio Money Expense Dashboard

Context — Jio Money (Reliance's digital wallet) shows users a monthly spending breakdown across categories — Recharge, Shopping, Food, Bills — in their app's home dashboard.

Challenge — Displaying the category-wise totals for a user's current month requires aggregating potentially hundreds of transaction rows into 5-6 summary numbers, fast enough to load within 200ms on a 4G connection.

What they did — The backend runs a single GROUP BY SQL query (similar to the one above) on an indexed transactions table. Results are cached in Redis for 5 minutes. JDBC reads the ResultSet and maps each row to a CategorySummary Java object, which is then serialized to JSON for the mobile app.

Result — Dashboard loads in under 150ms for 95% of users. The key was doing aggregation in SQL (not in Java loops), which is always faster because the database is optimized for set operations.

Takeaway — Do your math in SQL, not in Java. SUM(), GROUP BY, and ORDER BY in the query always outperform looping through rows in application code.

Source: Reliance Jio Money product documentation / general fintech architecture patterns

"The ResultSet is not a table — it is a one-way read head. You move forward through rows one at a time, read what you need, and close it. That discipline is what makes JDBC fast."

WHERE TO GO NEXT

You can now retrieve, format, and paginate data from MySQL using ResultSet. The next session brings in transactions — ensuring that multiple related queries succeed or fail together.

Explore deeper — MySQL EXPLAIN command documentation — learn to read query execution plans: <https://dev.mysql.com/doc/refman/8.0/en/explain.html>

Try on your own — Add a created_at DATETIME column to your expenses table (ALTER TABLE expenses ADD COLUMN created_at DATETIME DEFAULT NOW()). Then write a query that fetches expenses from the current month only and groups them by category. Print the output as a mini report.

Connect the dots — Session 05 covers transactions — what happens when a fund transfer needs two UPDATES and one of them fails halfway through.

Going further — Research 'JDBC RowMapper' in Spring JDBC — a design pattern for automatically mapping ResultSet rows to Java objects. It eliminates the boilerplate you wrote today and is used in virtually every Spring project.

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Week 9, Lecture 4: JDBC — Querying the Database

Link — <https://nptel.ac.in/courses/106105191>

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java tutor. Explain in simple Hinglish how a ResultSet cursor works, using the analogy of a bank passbook. Each page of the passbook is a row; you read one entry at a time from top to bottom and cannot skip back. Include an Indian banking example. Keep it under 150 words.

SESSION

05

60 MIN · DATA INTEGRITY

Transactions & Batch Processing

◆ Learning Objectives

- Explain what a database transaction is and why atomicity matters
- Disable auto-commit and manually call `commit()` to group queries atomically
- Use `rollback()` inside a catch block to undo failed transactions
- Describe all four ACID properties with one real-world example each
- Use `addBatch()` and `executeBatch()` to insert multiple rows in a single round-trip
- Calculate the performance advantage of batch inserts over single-row inserts

01 The Problem That Transactions Solve

Imagine a UPI transfer: Rs 500 is debited from Riya's account and credited to Arjun's account. This requires two UPDATE statements. What if the first UPDATE succeeds (Rs 500 deducted from Riya) but the server crashes before the second UPDATE (Rs 500 added to Arjun) runs? Riya has lost Rs 500 and Arjun never received it. Without transactions, this silent money destruction is possible. Transactions ensure that either both updates succeed together, or neither happens — no half-states.

CONCEPT — ACID Properties

Every database transaction must be ACID:

Atomicity — All queries in a transaction succeed or all are undone. No half-results.

Consistency — The database moves from one valid state to another. Balances cannot go negative.

Isolation — Concurrent transactions do not interfere. Two cashiers cannot update the same account simultaneously.

Durability — Once committed, the data survives power failures and crashes. It is safely written to disk.

02 Auto-Commit — The Default Behaviour

By default, every JDBC statement executes in its own transaction and is committed immediately — this is called auto-commit mode. For single-query operations (add a row, update a record) this is convenient. But for any operation that requires multiple queries to succeed together — like a transfer, an order placement, or a batch import — you must turn auto-commit off and manage the transaction yourself.

Auto-Commit ON (Default)

Every `executeUpdate()` is committed instantly. Safe for single-query operations. No explicit `commit()` call needed. Dangerous for multi-step operations — a crash halfway leaves partial data in the DB.

Auto-Commit OFF (Manual Transaction)

You call `conn.setAutoCommit(false)` once. Then you execute multiple queries. Only when you call `conn.commit()` are all changes written permanently. If anything goes wrong, `conn.rollback()` undoes all of them.

03 Writing a Safe Fund Transfer Transaction

```
conn.setAutoCommit(false);
try {
    // Step 1: Debit sender
    PreparedStatement debit = conn.prepareStatement("UPDATE wallets SET balance = balance
- ? WHERE user_id = ? AND balance >= ?");
    Deduct only if balance is sufficient
    debit.setDouble(1, 500.00);
    debit.setInt(2, 101);
    debit.setDouble(3, 500.00);
    int debitRows = debit.executeUpdate();
    if (debitRows == 0) throw new Exception("Insufficient balance");
    Abort if debit failed
    // Step 2: Credit receiver
    PreparedStatement credit = conn.prepareStatement("UPDATE wallets SET balance = balance
+ ? WHERE user_id = ?");
    Add to receiver
    credit.setDouble(1, 500.00);
    credit.setInt(2, 202);
    credit.executeUpdate();
    conn.commit();
    System.out.println("Transfer successful!");
} catch (Exception e) {
    conn.rollback();
    System.out.println("Transfer failed, rolled back: " + e.getMessage());
} finally {
    conn.setAutoCommit(true);
}
```

Turn off auto-commit — manual control begins

Wrap all transaction code in try

Amount to deduct

Sender's user_id

Minimum balance check in WHERE

Returns 0 if balance insufficient

Amount to add

Receiver's user_id

Credit the amount

Both steps succeeded — write permanently

Any error triggers rollback

Undo BOTH changes — database returns to original state

Always restore auto-commit

Reset for subsequent operations

End transaction block

TRY THIS LIVE

Create a wallets table: `CREATE TABLE wallets (user_id INT PRIMARY KEY, balance DECIMAL(10,2)).`

Insert two rows: (101, 1000.00) and (202, 500.00). Run the transfer code above. Verify that both balances updated correctly with a SELECT. Now deliberately throw an exception after the debit (add: if(1==1) throw new RuntimeException("Simulated crash");) and run again. Then SELECT both wallets — balances should be unchanged, proving rollback worked.

04 Batch Processing — Multiple Inserts in One Shot

Every executeUpdate() call is a network round-trip: Java sends a query to MySQL, MySQL processes it, MySQL sends back a result. For 1,000 individual inserts, that is 1,000 round-trips. On a local machine this takes under a second. On a cloud server in Mumbai with a database in a different data centre, latency accumulates and the same 1,000 inserts can take 20-30 seconds. JDBC batch processing bundles all 1,000 queries into a single network round-trip.

BATCH VS SINGLE-INSERT SPEED

10–50×

Typical speed improvement of JDBC batch inserts over individual executeUpdate() calls — Source: MySQL Connector/J performance benchmarks

```
String sql = "INSERT INTO expenses (user_id, category, amount) VALUES (?, ?, ?)";
```

SQL template — prepared once

```
PreparedStatement ps = conn.prepareStatement(sql);
```

Prepare

```
conn.setAutoCommit(false);
```

Batch + transaction = maximum safety

```
for (int i = 0; i < expenseList.size(); i++) {
```

Loop over all expenses

```
ps.setInt(1, expenseList.get(i).getUserId());
```

Bind user_id

```
ps.setString(2, expenseList.get(i).getCategory());
```

Bind category

```
ps.setDouble(3, expenseList.get(i).getAmount());
```

Bind amount

```
ps.addBatch();
```

Queue this row — does NOT send to DB yet

```
if (i % 500 == 0) {
```

Send in chunks of 500 — avoids memory overflow

```
ps.executeBatch();
```

Send all queued rows in one round-trip

```
ps.clearBatch();
```

Clear the queue

```
}
```

End loop

```
}
```

```
ps.executeBatch();
```

Send any remaining rows in the last partial chunk

```
conn.commit();
```

Commit all inserts as one transaction

CONCEPT — Chunked Batching

Never add all rows to the batch before calling `executeBatch()`. If you are inserting 1 lakh rows, holding them all in memory will `OutOfMemoryError` your JVM. Instead, flush the batch every 500 or 1,000 rows as shown above. This keeps memory consumption flat while still dramatically reducing round-trips compared to individual inserts.

05 `executeBatch()` Return Value

Return Value	Meaning	Action to Take
<code>int[] counts</code>	Array — one int per queued statement. Each int = rows affected.	Check all values are ≥ 0
<code>counts[i] = 1</code>	That insert/update succeeded normally	All good
<code>counts[i] = Statement.SUCCESS_NO_INFO (-2)</code>	MySQL processed it but won't say how many rows	Treat as success
<code>counts[i] = Statement.EXECUTE_FAILED (-3)</code>	That statement in the batch failed	Log and roll back if needed
<code>BatchUpdateException</code> thrown	One or more statements caused an error	Call <code>rollback()</code> in catch block

CASE STUDY — Flipkart's Big Billion Day Inventory Sync

Context — Flipkart (headquartered in Bengaluru, India's largest e-commerce platform) runs a Big Billion Day sale where millions of product inventory records are updated simultaneously.

Challenge — During sale preparation, inventory teams upload lakhs of new product records from seller spreadsheets. Inserting each row with individual JDBC calls took over 2 hours for a 5-lakh row dataset — too slow to run before sale go-live.

What they did — The data import pipeline was rewritten to use JDBC batch inserts with a chunk size of 1,000 rows, wrapped in a transaction per chunk. Each chunk commits independently so a failure in chunk 300 does not undo chunks 1-299.

Result — Import time dropped from 2 hours to under 8 minutes for the same 5-lakh row dataset. The chunked transaction model also gave the team visibility into partial failures without total rollback.

Takeaway — Batch processing is not an optimisation — for large data volumes it is the only feasible approach. Single-row inserts at scale are like delivering 1 lakh packages one auto-rickshaw at a time.

Source: Flipkart Tech Blog / general e-commerce data pipeline patterns

*“A transaction without a rollback plan is like a UPI transfer without a refund policy.
Always write your catch block before you write your commit.”*

WHERE TO GO NEXT

You can now group queries into atomic transactions and bulk-insert thousands of rows efficiently. The final session ties everything together with production-grade resource management.

Explore deeper — MySQL InnoDB transaction isolation levels:

<https://dev.mysql.com/doc/refman/8.0/en/innodb-transaction-isolation-levels.html>

Try on your own — Simulate the expense import feature for the Smart Expense Tracker mini project.

Generate a Java List of 200 random Expense objects (use a for loop with Math.random()) and batch-insert them into your expenses table. Measure the time with System.currentTimeMillis() before and after.

Connect the dots — Session 06 covers try-with-resources and DatabaseMetaData — the professional resource management and introspection tools that round out your JDBC skill set.

Going further — Research 'Spring @Transactional annotation' — it wraps the entire pattern you just built (setAutoCommit(false), commit(), rollback()) into a single annotation. Understanding what it does under the hood — which you now do — is what separates developers who use frameworks from developers who understand them.

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Week 9, Lecture 5: JDBC — Transactions and Batch Updates

Link — <https://nptel.ac.in/courses/106105191>

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a database trainer. Explain ACID properties in simple Hinglish using a GPay money transfer between two friends in Ahmedabad as the example. Make sure each of the four properties — Atomicity, Consistency, Isolation, Durability — is explained with one concrete detail from that transfer scenario. Keep it under 150 words.

SESSION

06

60 MIN · PRODUCTION PRACTICES

Integration & Error Handling

◆ Learning Objectives

- Use try-with-resources to automatically close JDBC objects without explicit finally blocks
- Explain why resource leaks are dangerous and how unclosed connections cause server failures
- Use DatabaseMetaData to inspect database and driver version information programmatically
- Use ResultSetMetaData to dynamically discover column names and types at runtime
- Structure a complete JDBC utility class (DBUtil) for reuse across a project
- Identify and fix the three most common JDBC resource leak patterns

01 The Resource Leak Problem

Every JDBC Connection, Statement, and ResultSet holds an open resource on both the Java side and the MySQL server side. If your code throws an exception between opening a Connection and closing it, the close() call in a finally block is the only safety net. If you forget the finally block — which is easy to do — the connection stays open. On a server running 24×7, this adds up: after a few days of intermittent errors, MySQL hits its max_connections limit and refuses all new connections. The app goes down.

WARNING — Classic Resource Leak Pattern

This is the most common JDBC mistake in junior code: `Connection conn = DBUtil.getConnection(); Statement st = conn.createStatement(); ResultSet rs = st.executeQuery(sql);` — and then an exception is thrown before `conn.close()`. The connection is leaked. Over hundreds of requests this fills MySQL's connection pool and crashes the application. The fix is try-with-resources, covered in this session.

02 try-with-resources — The Modern Solution

CONCEPT — AutoCloseable

Java 7 introduced the AutoCloseable interface. Any class that implements it can be declared inside try(...) parentheses. Java guarantees that its close() method is called automatically when the try block exits — whether by normal completion, a return statement, or an exception. Connection, PreparedStatement, and ResultSet all implement AutoCloseable. This means you can eliminate finally blocks entirely and the resources will always be released safely.

```
// OLD WAY — error-prone manual close
Connection conn = null;
PreparedStatement ps = null;
```

Pre-Java 7 pattern

```
try { conn = DBUtil.getConnection(); ... }
finally { if(conn != null) conn.close(); }
```

Easy to forget or get wrong

```
// MODERN WAY — try-with-resources
String sql = "SELECT * FROM expenses WHERE user_id = ?";

Query
try (Connection conn = DBUtil.getConnection();
    conn auto-closed when try exits
    PreparedStatement ps = conn.prepareStatement(sql)) {
    ps auto-closed too — inner first, then outer
    ps.setInt(1, 101);
    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            System.out.println(rs.getString("category"));
        }
    }
}
catch (SQLException e) { e.printStackTrace(); }
```

Java 7+ — always use this

Bind parameter

ResultSet in inner try-with-resources

Read rows

End while

rs.close() called automatically here

ps.close() then conn.close() called automatically here

Handle errors

TRY THIS LIVE

Refactor your Session 04 SELECT code to use try-with-resources. Remove all explicit `conn.close()`, `ps.close()`, and `rs.close()` calls. Run it — the output should be identical. Now deliberately add `throw new RuntimeException("test")` before `rs.next()`. Put a `System.out.println("Before close")` just before the exception and add `conn.isClosed()` as a final line after the try block. Confirm the connection is properly closed even though an exception was thrown.

03 Building a DBUtil Class

Every file in your project that needs a JDBC connection should not repeat the URL, username, password, and `DriverManager.getConnection()` call. Instead, centralise this in a `DBUtil` (Database Utility) class with a static `getConnection()` method. This single-responsibility class is the first step towards the DAO (Data Access Object) pattern used in all professional Java projects.

```
public class DBUtil {
    private static final String URL = System.getenv("DB_URL");
    private static final String USER = System.getenv("DB_USER");
    private static final String PASS = System.getenv("DB_PASS");
}
```

Utility class — no instances needed

Read from environment variable

Not hardcoded in source

Never commit passwords to git

```
public static Connection getConnection() throws SQLException {
```

Factory method — returns a fresh connection

```
return DriverManager.getConnection(URL, USER, PASS);
```

Caller must close it — use try-with-resources

```
    }  
}
```

All DAO classes call DBUtil.getConnection() — one place to change if DB moves

04 DatabaseMetaData — Inspecting the Database Itself

DatabaseMetaData is an interface that gives you information about the database server and driver — not about your data, but about the database system itself. It is useful for writing portable code, for health-check endpoints, and for debugging connection issues in production. You get it by calling `conn.getMetaData()`.

```
try (Connection conn = DBUtil.getConnection()) {
```

Open connection

```
DatabaseMetaData meta = conn.getMetaData();
```

Get metadata object from the connection

```
System.out.println("DB Product: " + meta.getDatabaseProductName());
```

Prints: MySQL

```
System.out.println("DB Version: " + meta.getDatabaseProductVersion());
```

Prints: 8.0.xx

```
System.out.println("Driver Name: " + meta.getDriverName());
```

Prints: MySQL Connector/J

```
System.out.println("Driver Version: " + meta.getDriverVersion());
```

Prints: mysql-connector-j-8.x

```
System.out.println("JDBC Version: " + meta.getJDBCMinorVersion() + "." +  
meta.getJDBCMinorVersion());
```

Prints: 4.2 (JDBC spec version)

```
System.out.println("Max Connections: " + meta.getMaxConnections());
```

MySQL's configured max_connections limit

05 ResultSetMetaData — Discovering Columns Dynamically

ResultSetMetaData describes the columns of a ResultSet — their names, types, and sizes. You use it when you do not know at compile time which columns a query will return. This is common in generic report generators, database admin tools, and CSV export utilities. You get it by calling `rs.getMetaData()` after running a query.

```
String sql = "SELECT * FROM expenses LIMIT 1";
```

Run any query

```
try (Connection conn = DBUtil.getConnection();
```

```
PreparedStatement ps = conn.prepareStatement(sql);
```

```
    ResultSet rs = ps.executeQuery()) {
```

```
    ResultSetMetaData rsMeta = rs.getMetaData();
```

Get column metadata

```
        int colCount = rsMeta.getColumnCount();
```

How many columns does this query return?

```
        for (int i = 1; i <= colCount; i++) {
```

Column indexes are 1-based in JDBC

```
        System.out.println("Col " + i + ": " + rsMeta.getColumnName(i)
```

Column name

```
        + " (" + rsMeta.getColumnTypeName(i) + ", " + rsMeta.getColumnDisplaySize(i) + ")");
```

Type name and display width

```
    }
```

Prints all column names and types dynamically

ResultSetMetaData Method	Returns	Useful For
getColumnCount()	int — number of columns	Driving loops over columns
getColumnName(i)	String — column name	Building CSV headers dynamically
getColumnTypeName(i)	String — SQL type e.g. VARCHAR, INT	Deciding which getter to use
getColumnDisplaySize(i)	int — max display width	Formatting column widths in a report
isNullable(i)	int — 0=NO, 1=YES, 2=UNKNOWN	Deciding whether null checks are needed
getPrecision(i)	int — total digits for DECIMAL	Formatting financial amounts correctly

06

Putting It All Together — A Generic CSV Exporter

The real power of `ResultSetMetaData` is building utilities that work with any table. The generic CSV export method below runs any `SELECT` query, discovers its column names dynamically using `ResultSetMetaData`, then streams every row to a file. This single method can export expenses, users, transactions — any table — without any table-specific code.

```
public static void exportToCSV(String sql, String filename) throws Exception {
```

Generic export — works for any query

```
try (Connection conn = DBUtil.getConnection();
```

```
PreparedStatement ps = conn.prepareStatement(sql);
```

```
    ResultSet rs = ps.executeQuery();
```

```

FileWriter fw = new FileWriter(filename)) {
    FileWriter also AutoCloseable
    ResultSetMetaData meta = rs.getMetaData();
    Get column info
        int cols = meta.getColumnCount();           Number of columns
        for (int i = 1; i <= cols; i++) {           Write header row
    fw.write(meta.getColumnName(i) + (i < cols ? "," : "\n"));
    Comma-separated headers
        }
        while (rs.next()) {                         Write each data row
            for (int i = 1; i <= cols; i++) {         Read each column
    fw.write((rs.getString(i) != null ? rs.getString(i) : "") + (i < cols ? "," : "\n"));
    Null-safe value + comma
            }
        }
    }
}
    All resources auto-closed
    Call: exportToCSV("SELECT * FROM expenses",
    "expenses.csv");

```

CASE STUDY — Razorpay Settlement Reports

Context — Razorpay, India's leading payment gateway (based in Bengaluru, serving over 8 million businesses), generates daily settlement CSV files for every merchant — showing which transactions were settled, when, and for how much.

Challenge — Merchants range from solo freelancers to enterprise retailers. Their settlement reports have different columns depending on their account type, payment methods enabled, and country. Writing a separate report generator for each merchant type was unsustainable.

What they did — Razorpay's reporting engine uses a metadata-driven approach: a single SQL query is built dynamically per merchant, executed via JDBC, and the `ResultSetMetaData` drives the CSV column headers automatically. The same code path generates reports for all merchant types — no hardcoded column names anywhere in the export layer.

Result — Adding a new column to merchant settlements (e.g., international settlement column) requires only a schema change and query update — the export code requires zero modification. This saved engineering time on every billing cycle change.

Takeaway — Metadata-driven code is not just clever — it is maintainable. When your code adapts to the schema rather than hardcoding it, one developer can manage hundreds of report types.

Source: [Razorpay engineering blog / general payment platform architecture patterns](#)

“Closing a JDBC connection is not optional — it is a contract with the database. try-with-resources is how you honour that contract even when things go wrong.”

WHERE TO GO NEXT

You now have a complete, production-aware JDBC toolkit: connection management, parameterised queries, transactions, batch processing, and metadata introspection. You are ready to build the Smart Expense Tracker mini project.

Explore deeper — Spring JDBC's JdbcTemplate source code on GitHub — see how it wraps everything you learned: <https://github.com/spring-projects/spring-framework>

Try on your own — Build a complete DBUtil class for your Smart Expense Tracker project. Add a static printDBInfo() method that uses DatabaseMetaData to print the DB name, version, and driver. Call it at application startup as a health check.

Connect the dots — Module 3 (Web Technologies in Java) builds on this JDBC foundation — your Servlet will call the same DAO classes you have been writing, now serving data to a web browser instead of a console.

Going further — Research HikariCP connection pooling. It replaces DriverManager.getConnection() with a pool that pre-creates connections and reuses them. Understand why it changes DBUtil.getConnection() from a factory method into a pool checkout — and why this matters in a web server handling 1,000 requests per second.

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Week 9, Lecture 6: JDBC — Metadata and Advanced Features

Link — <https://nptel.ac.in/courses/106105191>

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java tutor. Explain in simple Hinglish what happens when a JDBC connection is not closed — use the analogy of leaving a water tap open in an Ahmedabad flat. Then explain how try-with-resources is like an automatic tap that closes itself. Include one code example. Keep it under 150 words.

REVIEW



INTERVIEW PREP · MODULE 2

Interview Questions

◆ RDBMS & Database Programming With JDBC

Rehearse these questions out loud, aiming for a 60-second answer each. The hint is only a nudge — try answering before you look at it. Compare your answer to the model answer afterward, not before.

1

What is JDBC and why do we need it in Java applications?

Think about how Java interacts with databases.

MODEL ANSWER

JDBC (Java Database Connectivity) is a Java API in the `java.sql` package that allows Java programs to connect to and interact with relational databases. Without JDBC, Java has no built-in way to talk to a MySQL, PostgreSQL, or Oracle database. JDBC provides a standard set of interfaces so that the same Java code can work with different databases by simply swapping the driver. It enables Java applications to execute SQL queries, retrieve results, and manage transactions from within Java code.

2

Explain the main components of JDBC architecture.

Focus on Driver Manager, Connection, and Statement.

MODEL ANSWER

The main components are: `DriverManager` (manages database drivers and establishes connections), `Connection` (represents an active session with the database), `Statement` / `PreparedStatement` (used to send SQL commands to the database), `ResultSet` (holds the data returned by a `SELECT` query), and the JDBC Driver itself (database-specific implementation, e.g., `MySQL Connector/J`). The flow is: load driver -> get connection via `DriverManager` -> create a statement -> execute SQL -> process `ResultSet` -> close resources.

3

In a typical Java application, how does JDBC connect to a MySQL database?**MODEL ANSWER**

You provide a connection URL in the format `jdbc:mysql://hostname:port/databaseName`, along with a username and password, to `DriverManager.getConnection()`. The MySQL JDBC driver (connector JAR) must be on the classpath. From Java 6+, the driver class is auto-loaded, but you can explicitly load it with `Class.forName('com.mysql.cj.jdbc.Driver')`. `DriverManager` returns a `Connection` object that represents a live session, which you then use to create statements and run queries.

4

What is the purpose of the MySQL connector JAR file when working with JDBC?

Consider how Java communicates with MySQL.

MODEL ANSWER

JDBC itself is just a set of interfaces — it defines what operations are possible but not how they work for a specific database. The MySQL Connector/J JAR (e.g., `mysql-connector-java-8.x.x.jar`) is the MySQL-specific implementation of those interfaces. It contains the actual code that translates JDBC calls into MySQL's native protocol. Without it, Java cannot communicate with MySQL. You add it to the project's classpath (or as a Maven/Gradle dependency), and JDBC's `DriverManager` detects and uses it automatically.

5

Describe the steps to establish a database connection using JDBC in Java.

MODEL ANSWER

1) Add the MySQL Connector/J JAR to the project classpath. 2) Import `java.sql.*` in your Java file. 3) Define the connection URL, username, and password. 4) Call `Connection con = DriverManager.getConnection(url, user, password);` 5) Check that the connection is not null, indicating success. 6) Use the connection to create statements. 7) Close the connection in a finally block or use `try-with-resources` to avoid resource leaks. Error handling with `try-catch` around the `getConnection()` call is essential since it throws a checked `SQLException`.

6

Write a short Java code snippet to connect to a MySQL database using `DriverManager.getConnection()`.

Remember to include the connection URL, username, and password.

MODEL ANSWER

```
String url = "jdbc:mysql://localhost:3306/expense_db"; String user = "root"; String pass = "password"; try
(Connection con = DriverManager.getConnection(url, user, pass)) { System.out.println("Connected
successfully!"); } catch (SQLException e) { System.out.println("Connection failed: " + e.getMessage()); } —
The try-with-resources block ensures the connection is automatically closed after use. The URL includes the
protocol (jdbc:mysql://), host, port, and database name.
```

7

How should you handle exceptions when establishing a JDBC connection?

MODEL ANSWER

JDBC's `getConnection()` throws a checked `SQLException`, so it must be handled. Wrap the connection code in a `try-catch (SQLException e)` block. Log or print `e.getMessage()` for debugging and provide a user-friendly error message. Use `try-with-resources (try (Connection con = ...))` to guarantee the connection is closed even if an exception occurs inside the `try` block. In production, do not expose raw SQL error messages to users as they may reveal database structure. Consider logging with a framework like `SLF4J` and rethrowing as a custom application exception.

8

What is the difference between Statement and PreparedStatement in JDBC?

Think about performance and security aspects.

MODEL ANSWER

Statement executes a static SQL string directly: `stmt.execute("SELECT * FROM users");` It is simple but prone to SQL injection when user input is concatenated into the query. PreparedStatement pre-compiles the SQL with placeholders (?): `PreparedStatement ps = con.prepareStatement("SELECT * FROM users WHERE id = ?"); ps.setInt(1, userId);` — It is more secure because parameters are escaped automatically, and it is faster when the same query is executed multiple times since the database caches the execution plan. Always prefer PreparedStatement when queries include user-supplied data.

9

Why is PreparedStatement preferred over Statement for executing queries that include user input?

Consider SQL injection risks.

MODEL ANSWER

When you concatenate user input directly into a SQL string (e.g., `"SELECT * FROM users WHERE name = " + name + ""`), a malicious user can inject SQL code — for example, entering name as `' OR '1'='1` to bypass authentication. PreparedStatement uses parameterized queries with ? placeholders, and the JDBC driver ensures all parameters are treated as data, not SQL code. This completely prevents SQL injection. It also improves performance through query plan caching and makes code cleaner to read.

10

Write a Java code snippet using PreparedStatement to insert a new record into an 'expenses' table.

MODEL ANSWER

```
String sql = "INSERT INTO expenses (user_id, category, amount, date) VALUES (?, ?, ?, ?)"; try
(PreparedStatement ps = con.prepareStatement(sql)) { ps.setInt(1, userId); ps.setString(2, "Food");
ps.setDouble(3, 250.0); ps.setDate(4, Date.valueOf(LocalDate.now())); int rows = ps.executeUpdate();
System.out.println(rows + " row(s) inserted."); } catch (SQLException e) { e.printStackTrace(); } —
executeUpdate() returns the number of affected rows. Each ? is bound by its position (1-indexed) using the
appropriate setter method.
```

11

How can you update or delete a record in a database using PreparedStatement?

MODEL ANSWER

For update: `String sql = "UPDATE expenses SET amount = ? WHERE id = ?"; PreparedStatement ps = con.prepareStatement(sql); ps.setDouble(1, 300.0); ps.setInt(2, expenseld); ps.executeUpdate();` — For delete: `String sql = "DELETE FROM expenses WHERE id = ?"; ps.setInt(1, expenseld); ps.executeUpdate();` — Both use `executeUpdate()` which returns the number of rows affected. Check the return value — if it is 0, the record with the given ID was not found. Always use parameterized queries to prevent injection.

12

What is SQL Injection, and how does using PreparedStatement help prevent it?

Think about how user input is handled.

MODEL ANSWER

SQL Injection is an attack where a user enters malicious SQL code as input, which gets executed by the database. For example, entering ' OR '1'='1' -- in a login form could make a query return all users, bypassing authentication. PreparedStatement prevents this by treating all parameter values as literal data, not as SQL syntax. The JDBC driver escapes special characters like quotes, so malicious input cannot alter the query structure. This is the most critical security practice in any application that uses a database.

13

How do you execute a SELECT query using JDBC and process the ResultSet to display results?

MODEL ANSWER

```
PreparedStatement ps = con.prepareStatement("SELECT id, category, amount FROM expenses WHERE user_id = ?");
ps.setInt(1, userId);
ResultSet rs = ps.executeQuery();
while (rs.next()) {
    int id = rs.getInt("id");
    String cat = rs.getString("category");
    double amt = rs.getDouble("amount");
    System.out.println(id + " | " + cat + " | " + amt);
}
rs.close();
```

— executeQuery() returns a ResultSet. rs.next() moves the cursor to the next row and returns false when there are no more rows. Access columns by name or index using typed getters.

14

Write a Java method that retrieves all expenses for a given user from the database and prints them in a formatted way.

MODEL ANSWER

```
public void printExpenses(Connection con, int userId) throws SQLException {
    String sql = "SELECT category, amount, date FROM expenses WHERE user_id = ? ORDER BY date DESC";
    try (PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setInt(1, userId);
        ResultSet rs = ps.executeQuery();
        System.out.printf("%-15s %-10s %-12s\n", "Category", "Amount", "Date");
        while (rs.next()) {
            System.out.printf("%-15s %-10.2f %-12s\n", rs.getString("category"), rs.getDouble("amount"), rs.getDate("date"));
        }
    }
}
```

— printf formatting aligns columns for readability. ORDER BY date DESC shows most recent first.

15

What is a ResultSet in JDBC? How do you traverse it?

MODEL ANSWER

A ResultSet is an object returned by executeQuery() that holds the rows resulting from a SELECT query. It maintains a cursor initially positioned before the first row. You traverse it using a while loop with rs.next(), which moves the cursor forward and returns true as long as there are more rows. Access the current row's columns using typed getter methods: rs.getInt(columnName), rs.getString(columnName), rs.getDouble(columnName), etc. Always close the ResultSet after use (or use try-with-resources). By default, a ResultSet is forward-only and read-only.

16

Explain how you would implement pagination and sorting when displaying results fetched from a database.

Consider using SQL LIMIT/OFFSET and ORDER BY.

MODEL ANSWER

In the SQL query, add ORDER BY to sort (e.g., ORDER BY date DESC) and LIMIT/OFFSET for pagination: SELECT * FROM expenses WHERE user_id = ? ORDER BY date DESC LIMIT ? OFFSET ? — Set LIMIT to the page size (e.g., 10) and OFFSET to (pageNumber - 1) * pageSize. In Java, pass these as PreparedStatement parameters. Maintain the current page number in your application state. For navigation, increment or decrement the page number and re-execute the query. The total number of pages can be calculated from a COUNT(*) query.

17

What is auto-commit mode in JDBC, and how does it affect transactions?

Think about when changes are saved to the database.

MODEL ANSWER

By default, JDBC connections have auto-commit mode enabled, meaning every SQL statement is immediately committed to the database as its own transaction. If you execute an INSERT, it is saved instantly. This is convenient but dangerous for operations that must succeed or fail together (like a bank transfer debit + credit). To manage transactions manually, you disable auto-commit with con.setAutoCommit(false), execute your statements, and then call con.commit() to save all changes, or con.rollback() to undo them if an error occurs.

18

How do you manually manage transactions in JDBC using commit() and rollback()?

MODEL ANSWER

```
con.setAutoCommit(false); try { ps1.executeUpdate(); // debit source account ps2.executeUpdate(); // credit
target account con.commit(); System.out.println("Transfer successful."); } catch (SQLException e) {
con.rollback(); System.out.println("Transfer failed, rolled back."); } finally { con.setAutoCommit(true); } —
```

Disabling auto-commit groups both updates into one atomic transaction. If either fails, rollback() undoes both. This guarantees that either both changes are applied or neither is, maintaining data integrity.

19

Write a code snippet to insert multiple expense records in a single batch using JDBC.

MODEL ANSWER

```
con.setAutoCommit(false); PreparedStatement ps = con.prepareStatement("INSERT INTO expenses
(user_id, category, amount) VALUES (?, ?, ?)"); for (Expense e : expenseList) { ps.setInt(1, e.getUserId());
ps.setString(2, e.getCategory()); ps.setDouble(3, e.getAmount()); ps.addBatch(); } ps.executeBatch();
con.commit(); — addBatch() queues each set of parameters. executeBatch() sends them all in one database
round-trip. Disable auto-commit so all inserts are part of one transaction; rollback if the batch fails.
```

20

Why is batch processing useful when inserting large numbers of records?

MODEL ANSWER

Without batching, each INSERT statement requires a separate round-trip from the Java application to the database — this is slow for hundreds or thousands of records due to network latency and query parsing overhead for each call. With batch processing, multiple statements are grouped and sent to the database in a single network call. `executeBatch()` processes them all server-side. This can reduce insertion time dramatically — from minutes to seconds for large datasets. It is essential for bulk data imports, log processing, and ETL operations.

21

If an error occurs during a batch insert, how can you ensure that none of the records are saved to the database?

MODEL ANSWER

Set auto-commit to false before executing the batch. Wrap `executeBatch()` in a try-catch. If a `BatchUpdateException` or `SQLException` is caught, call `con.rollback()`. This undoes all previously executed statements in the batch within the transaction. Without this, some records might be inserted before the error, leading to partial data. After rollback, log the error and decide whether to retry the entire batch or report the failure to the user. Always restore auto-commit state in the finally block.

22

How does the try-with-resources statement help with resource management in JDBC?

Think about closing connections and statements automatically.

MODEL ANSWER

JDBC resources (`Connection`, `Statement`, `ResultSet`) implement the `AutoCloseable` interface, so they can be used in try-with-resources: `try (Connection con = DriverManager.getConnection(url, user, pass); PreparedStatement ps = con.prepareStatement(sql)) { ... }` — Java automatically calls `close()` on each resource when the try block exits, whether normally or due to an exception. This prevents connection leaks, which can exhaust the database's connection pool. Before Java 7, developers had to manually close resources in finally blocks, which was error-prone.

23

What are the risks of not closing JDBC connections properly?

MODEL ANSWER

Databases have a limited number of simultaneous connections. If a Java application opens connections but never closes them, it exhausts the database connection pool — new connection requests will fail with 'too many connections' errors. This can crash the application under load. Additionally, open connections consume memory and file descriptors on both the Java and database sides. Connection leaks are one of the most common and damaging bugs in JDBC applications. Always close connections in finally blocks or use try-with-resources.

24

Explain the difference between DatabaseMetaData and ResultSetMetaData in JDBC.*One gives info about the database, the other about the result set.***MODEL ANSWER**

DatabaseMetaData provides information about the database itself — its product name, version, supported features, list of tables, schemas, and stored procedures. Obtained via: `DatabaseMetaData meta = con.getMetaData();` ResultSetMetaData provides information about the columns in a specific ResultSet — the number of columns, each column's name, data type, and display size. Obtained via: `ResultSetMetaData rsmeta = rs.getMetaData();` Use DatabaseMetaData for database introspection and ResultSetMetaData for dynamically processing query results without knowing the schema in advance.

25

How would you use exception handling to make your JDBC code more robust?**MODEL ANSWER**

Wrap all JDBC calls in try-catch (`SQLException e`) blocks since most JDBC methods throw checked `SQLExceptions`. Log the error with full details: `e.getMessage()`, `e.getSQLState()`, and `e.getErrorCode()`. In transaction code, catch the exception and call `rollback()`. Use multi-catch to handle different exceptions differently. Use try-with-resources to guarantee resource closure. In service or DAO layers, catch `SQLException` and wrap it in a custom application exception so the calling layer does not depend on JDBC-specific error types. Never swallow exceptions silently.

26

How can AI tools like GitHub Copilot assist you in writing JDBC code templates, and what should you check before using its suggestions?*Think about code correctness and security.***MODEL ANSWER**

AI tools can generate boilerplate JDBC code quickly — connection setup, prepared statements, ResultSet traversal, and DAO class templates. This saves significant typing time. Before using AI-generated JDBC code, verify: that `PreparedStatement` is used (not `Statement` with string concatenation) to prevent SQL injection; that try-with-resources or proper finally blocks close resources; that the SQL matches your actual table and column names; that parameter indices are correct (JDBC uses 1-based indexing); and that transactions are handled correctly for multi-step operations. Test with real data to confirm the code works as expected.

27

Describe the overall architecture of a mini project that integrates Java, JDBC, and SQL for an expense tracker application.**MODEL ANSWER**

The architecture follows a layered pattern: the Presentation layer handles console I/O and menus; the Business/Service layer contains logic like budget checking and report generation; the DAO (Data Access Object) layer handles all database operations using JDBC — separate DAOs for User and Expense; and the Database layer is MySQL with tables for users and expenses. The User class and Expense class are model/POJO objects that carry data between layers. JDBC connections can be managed via a `ConnectionManager` utility class. This separation makes each layer independently testable and maintainable.

28

How would you implement user registration and login functionality in a console-based Java application using JDBC?

MODEL ANSWER

For registration: read name, email, and password from console input; hash the password (or store plain text for a student project); call a `UserDAO.register()` method that executes `INSERT INTO users(name, email, password)` using a `PreparedStatement`; handle duplicate email exceptions. For login: read email and password; call `UserDAO.findByEmail()` which executes `SELECT * FROM users WHERE email = ?`; if a row is returned, compare the entered password with the stored one; if they match, return the `User` object and store it as the 'current user' for the session.

29

Write a Java method to add a new expense for a user, including input validation and error handling.

MODEL ANSWER

```
public void addExpense(Connection con, int userId, String category, double amount) throws SQLException {
    if (amount <= 0) throw new IllegalArgumentException("Amount must be positive.");
    if (category == null || category.isBlank()) throw new IllegalArgumentException("Category required.");
    String sql = "INSERT INTO expenses (user_id, category, amount, date) VALUES (?, ?, ?, CURDATE())";
    try (PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setInt(1, userId);
        ps.setString(2, category);
        ps.setDouble(3, amount);
        ps.executeUpdate();
    }
    // Validation happens before the DB call to avoid unnecessary queries.
}
```

30

How would you retrieve and display a monthly expense report grouped by category using JDBC and SQL?

Consider using SQL GROUP BY and date filtering.

MODEL ANSWER

```
String sql = "SELECT category, SUM(amount) AS total FROM expenses WHERE user_id = ? AND MONTH(date) = ? AND YEAR(date) = ? GROUP BY category ORDER BY total DESC";
PreparedStatement ps = con.prepareStatement(sql);
ps.setInt(1, userId);
ps.setInt(2, month);
ps.setInt(3, year);
ResultSet rs = ps.executeQuery();
while (rs.next()) {
    System.out.printf("%-15s %.2f%n", rs.getString("category"), rs.getDouble("total"));
}
// GROUP BY aggregates amounts per category, SUM() calculates totals, and the date filters restrict to the selected month.
```

31

What is the MVC (Model-View-Controller) pattern, and how does it apply to your Smart Expense Tracker project?

Think about separating database logic, user interface, and business logic.

MODEL ANSWER

MVC separates an application into three layers. Model: Java classes (`User`, `Expense`) that represent data, and DAO classes that interact with the database. View: the console menus and formatted output that the user sees. Controller: the main application logic that takes user input, calls the appropriate service or DAO methods, and directs the View to display results. In the expense tracker, `main()` acts as a controller — it reads user input, calls `ExpenseService.addExpense()`, and then prints a confirmation. This separation means UI changes do not affect database logic and vice versa.

32

If a user tries to delete an expense that doesn't exist, how should your application handle this scenario gracefully?

MODEL ANSWER

`executeUpdate()` returns the number of rows affected by the DELETE statement. If the expense ID does not exist, it returns 0. Check: `int rows = ps.executeUpdate(); if (rows == 0) { throw new ExpenseNotFoundException("Expense not found: " + id); }` — Catch this in the calling layer and display a user-friendly message: 'No expense found with that ID. Please check and try again.' Do not let the user see a raw SQL error. Logging the attempt helps in debugging and auditing.

33

Suppose your application is showing incorrect totals in the monthly report. What steps would you take to debug and fix the issue?

MODEL ANSWER

First, run the same SQL query directly in a MySQL client (MySQL Workbench or CLI) with the same parameters to confirm whether the issue is in Java or SQL. Check the date filtering — ensure `MONTH()` and `YEAR()` are being used correctly and that the date column is a proper DATE type, not a string. Verify that all relevant records exist in the database for that month. Add print statements or a debugger to log the SQL string and parameter values before execution. Check if there are duplicate rows or null amounts that could distort the sum. Fix and re-run.

34

How would you validate user input to prevent invalid data from being saved to the database in your expense tracker app?

MODEL ANSWER

Validate before calling the DAO: check that `amount > 0`, category is not null or empty, date is not null, and string fields do not exceed column length limits. Use try-catch around `Integer.parseInt()` and `Double.parseDouble()` to handle invalid number formats entered by the user. At the database level, add constraints (NOT NULL, CHECK, FOREIGN KEY) in the schema as a safety net. Return descriptive error messages so users know what to fix. Never rely solely on database constraints — validate on the Java side first for a better user experience.

35

Describe how you would integrate an 'AI Tips' section that fetches budgeting advice from an external API like ChatGPT in your Java application.

Think about making HTTP requests and displaying responses.

MODEL ANSWER

Use Java's `HttpClient` (Java 11+) to make a POST request to the OpenAI API endpoint, passing an authorization header with your API key and a JSON body containing the model name and prompt (e.g., 'Give me a budgeting tip for someone who spent most on Food this month.'). Parse the JSON response using a library like Jackson or Gson to extract the message content. Display the tip in the console. Store the API key in a configuration file, not hardcoded. Add error handling for network failures or API rate limits.

36

If you used Copilot or ChatGPT to generate JDBC code for your project, how would you verify that the generated code is correct and secure?

MODEL ANSWER

Check that PreparedStatement is used with ? placeholders — never String concatenation in SQL. Verify that resources (Connection, Statement, ResultSet) are closed properly via try-with-resources. Confirm that the table and column names match your actual schema. Test each DAO method with real data: insert a record and verify it appears in the DB, delete and confirm it is gone, update and check the new value. Test with invalid inputs to confirm validation and exceptions work. Run the full application flow to catch integration issues the AI might have missed.

37

Can you explain how exception handling and transaction management work together in JDBC to ensure data integrity?

Think about rolling back transactions when exceptions occur.

MODEL ANSWER

Exception handling catches errors when they occur; transaction management ensures those errors do not leave the database in an inconsistent state. The pattern is: disable auto-commit, execute multiple related SQL statements, and if any throws a SQLException, catch it and call rollback() — undoing all changes from that transaction. If all succeed, call commit(). This ensures atomicity: either all changes are saved or none are. For example, in a fund transfer, if the debit succeeds but the credit fails, rollback() ensures the debited amount is also reversed.

38

In your mini project, how did you structure your code to separate database access logic from business logic and user interface?

MODEL ANSWER

The project followed a three-layer architecture. Model package: User and Expense POJOs. DAO package: UserDao and ExpenseDao classes — each method handles one SQL operation (insert, update, delete, select) using JDBC, taking model objects as parameters. Service package: ExpenseService validates inputs, applies business rules (e.g., budget alerts), and calls DAOs. Main/View: the console menu reads input, calls service methods, and prints output. This means if the database changes, only the DAO layer needs updating; if business rules change, only the service layer changes.

39

Write a Java function that updates multiple expense records in a transaction, ensuring that either all updates succeed or none are applied.

MODEL ANSWER

```
public void bulkUpdate(Connection con, List<Expense> expenses) throws SQLException {
    con.setAutoCommit(false); try { PreparedStatement ps = con.prepareStatement("UPDATE expenses SET
    amount = ? WHERE id = ?"); for (Expense e : expenses) { ps.setDouble(1, e.getAmount()); ps.setInt(2,
    e.getId()); ps.addBatch(); } ps.executeBatch(); con.commit(); } catch (SQLException e) { con.rollback(); throw
    e; } finally { con.setAutoCommit(true); } } — Batching the updates and wrapping in a transaction ensures
    atomicity across all records.
```

40

How would you use `ResultSetMetaData` to dynamically display column names and values when printing query results?

Think about iterating over columns in the `ResultSet`.

MODEL ANSWER

```
ResultSet rs = ps.executeQuery(); ResultSetMetaData rsmeta = rs.getMetaData(); int cols = rsmeta.getColumnCount(); for (int i = 1; i <= cols; i++) { System.out.print(rsmeta.getColumnName(i) + "\t"); } System.out.println(); while (rs.next()) { for (int i = 1; i <= cols; i++) { System.out.print(rs.getString(i) + "\t"); } System.out.println(); } — This approach works for any table without hardcoding column names, useful for generic reporting tools.
```

41

Suppose you need to support a different database (e.g., PostgreSQL) in your Java application. What changes would you need to make in your JDBC setup?

MODEL ANSWER

Swap the driver JAR: replace `mysql-connector-java` with `postgresql-42.x.x.jar`. Update the connection URL: from `jdbc:mysql://localhost:3306/db` to `jdbc:postgresql://localhost:5432/db`. Change the username/password if they differ. Some SQL syntax may need adjustment — for example, PostgreSQL uses `SERIAL` instead of `AUTO_INCREMENT`. The JDBC API itself (`PreparedStatement`, `ResultSet`, etc.) stays the same. This is the power of JDBC's abstraction — most Java code is database-independent; only the driver, URL, and potentially some SQL dialects change.